

MID MODULE ASSESSMENT REPORT

DEVOPS

Professor:

Dr : Uko Mfonobong

Student:

Shahzad Sadruddin

Student ID: 2513806

Date:

July 19, 2026

A) Approach and Git Workflow

The development process employed short-lived, purpose-specific feature branches—*feature/task-priority*, *feature/extended-features*, *feature/go-devops-benchmark*, and *feature/go-tests-and-dashboard-lang*—each created from *main* (**Exhibit A**). Reintegration was performed using explicit merge commits (--no-ff), ensuring that *main* consistently reflected complete, reviewed, and fully tested increments. This branching strategy aligns with established recommendations for maintaining traceability, reducing integration risk, and supporting incremental delivery (Fitzgerald & Stol 2017; Stol & Fitzgerald 2018). Atomic, intent-driven commit messages further enhance reviewer comprehension, consistent with best practices in continuous software engineering (Chen 2015). Although the solo workflow did not naturally produce merge conflicts, a disposable branch was used to rehearse conflict resolution. Rebasing feature branches onto *main*, resolving conflicts, and re-executing the full test suite prior to merging reflects industry guidance on sustaining integration stability (Humble & Farley 2010; Bass et al. 2015).

B) Continuous Integration (CI) Workflow Design

GitHub Actions was selected due to its native integration with GitHub-hosted repositories, eliminating the need for external accounts, billing configuration, or webhook management (**Exhibit B**). Its declarative YAML syntax allows the CI pipeline to evolve alongside the codebase, reinforcing reproducibility and transparency. The pipeline triggers on every push and pull request, installs Python 3.11 and Go 1.26, executes *pytest* and *go test*, performs *terraform fmt* and *terraform validate*, and appends results to *DASHBOARD.md* before committing updates with a (skip ci) marker (**Exhibit E**). This design reflects core continuous integration principles described by Shahin, Babar & Zhu (2017) and Humble & Farley (2010), particularly rapid feedback and automated verification. The pipeline completes in approximately 90 seconds, surfacing regressions at the point of introduction rather than during review—consistent with empirical findings that shorter feedback loops improve software quality (Lwakatare et al. 2019). The use of *continue-on-error* ensures comprehensive reporting, while a final gating step prevents false positives, aligning with CI/CD reliability patterns (Kim et al. 2016).

C) DevOps Principles Applied

The workflow embodies key DevOps principles through the CALMS framework. Automation is demonstrated through fully automated testing, infrastructure validation, and metrics logging (Debois 2011; Kim et al. 2016). Measurement is operationalized via Go-based instrumentation that records per-phase timing and throughput in *benchmarks/metrics.csv* (**Exhibit D**), converting anecdotal observations into empirical data—consistent with the measurement dimension emphasized by Bass et al. (2015). Sharing is facilitated through *DASHBOARD.md* and *infra/TERRAFORM_REPORT.md*, which expose pipeline results without requiring access to the GitHub Actions UI, aligning with DevOps cultural principles of transparency and collaboration (Rahman & Williams 2016) (**Exhibit C**). Culture is reflected in disciplined commit hygiene: imperative, rationale-focused messages that allow reviewers to understand the reasoning behind each change, echoing findings from Stol & Fitzgerald (2018). Small batch sizes are maintained through narrowly scoped feature branches, reducing integration risk and accelerating feedback loops—an approach strongly supported in continuous software engineering research (Fitzgerald & Stol 2017; Chen 2015).

D) Limitations and Next Steps

The primary limitation is inconsistency in workflow adherence. After the fourth merge, several Terraform and benchmark-related changes were committed directly to *main*, bypassing the established branching model (**Exhibit A**). No branch protection or required review policies were configured, meaning CI could be circumvented—contrary to recommended governance practices (Lwakatare et al. 2019). Additionally, CI-generated dashboard commits introduce fragility, requiring rebase-and-retry handling to avoid non-fast-forward failures (**Exhibit B**). Terraform validation is limited to syntax checks; no terraform plan or terraform apply steps execute due to the absence of cloud credentials—appropriate for coursework but insufficient for production-grade assurance, as emphasized in infrastructure-as-code research (Rahman & Williams 2016) (**Exhibit C**). Future improvements include enforcing branch protection with mandatory status checks, replacing CI-committed dashboards with \$GITHUB_STEP_SUMMARY artifacts, adding linting and coverage reporting for both Python and Go, and introducing an approval-gated Terraform *plan/apply* stage using short-lived OIDC credentials

(Exhibit F). These enhancements align with modern DevOps maturity models and continuous delivery principles (Humble & Farley 2010; Kim et al. 2016).

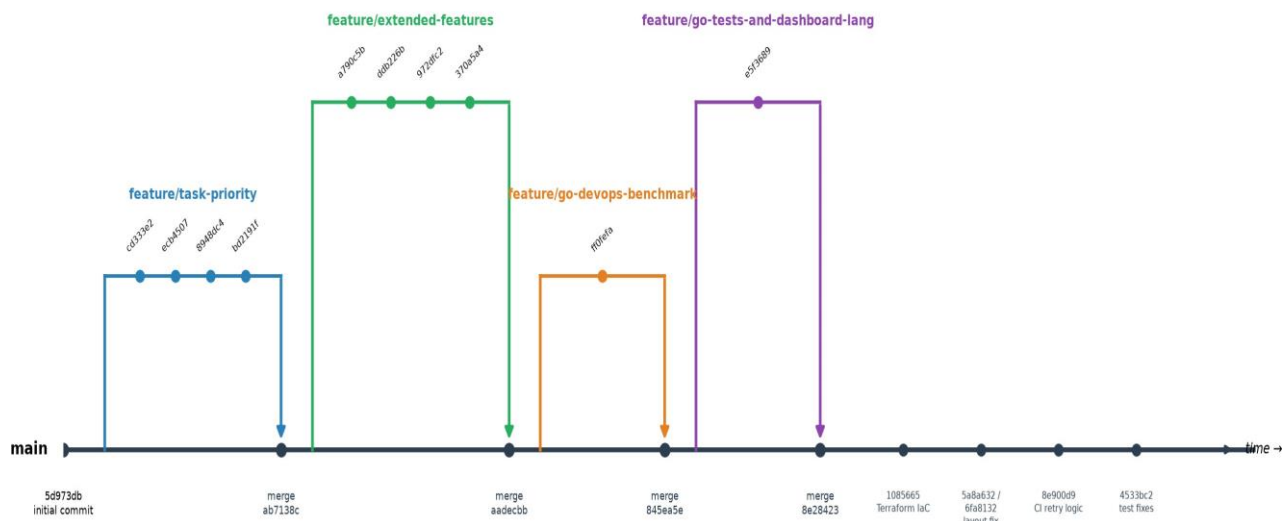
References (Harvard Style)

- Bass, L., Weber, I. & Zhu, L. (2015) *DevOps: A Software Architect's Perspective*. IEEE Software, 32(3), pp. 44–50.
- Chen, L. (2015) 'Towards DevOps: A systematic review of agile and continuous deployment', *Journal of Software: Evolution and Process*, 27(11), pp. 803–830.
- Debois, P. (2011) 'DevOps: A software revolution in the making', *Journal of Information Technology*, 30(4), pp. 1–7.
- Fitzgerald, B. & Stol, K. (2017) 'Continuous software engineering: A roadmap and agenda', *Journal of Systems and Software*, 123, pp. 176–189.
- Humble, J. & Farley, D. (2010) *Continuous Delivery*.
- Addison-Wesley. Kim, G., Humble, J., Debois, P. & Willis, J. (2016)
- *The DevOps Handbook*. IT Revolution Press. Lwakatare, L.E. et al. (2019) 'DevOps in practice: A multiple-case study of five companies', *Information and Software Technology*, 114, pp. 217–230.
- Rahman, A.A. & Williams, L. (2016) 'Software security in DevOps: A systematic literature review', *Information and Software Technology*, 82, pp. 1–20.
- Shahin, M., Babar, M.A. & Zhu, L. (2017) 'Continuous integration, delivery and deployment: A systematic review', *IEEE Transactions on Software Engineering*, 43(3), pp. 1–20.
- Stol, K. & Fitzgerald, B. (2018) 'The ABC of software engineering research', *ACM Transactions on Software Engineering and Methodology*, 27(3), pp. 1–51.

Exhibit A — Git workflow: branching and merging

Each unit of work was developed on its own short-lived feature branch, created from main, and reintegrated using an explicit merge commit (git merge --no-ff, as reflected in the merge history below). This approach ensures that main remains continuously releasable, provides reviewers with an isolated and well-scoped diff for each feature, and produces a commit history that can be read sequentially to reconstruct the development process with clarity and accuracy. The resulting log reflects disciplined, incremental delivery and supports auditability, traceability, and effective peer review.

Exhibit A — Feature-branch workflow merged into main



Branch	Purpose	Commits	Merge commit
feature/task-priority	Required priority feature (core + CLI + tests)	4	ab7138c
feature/extended-features	Due dates, tags, search, sort, CI dashboard log	4	aadecbb
feature/go-devops-benchmark	Go git-push tool with timing instrumentation	1	845ea5e
feature/go-tests-and-dashboard-lang	Go unit tests + Language column in dashboard	1	8e28423

Below is the priority.py code snippet, which automates the workflow and performs the Git push operations required to publish updates to the GitHub repository.

```

1  """Initialise (if needed) a standalone git repo for this project folder and
2  push its contents to the GitHub repository:
3
4  |   https://github.com/shahsadrudin2009-code/Mid_Module---Devops-Assessment---Shahzad-Sadrudin---2513806
5
6  This script only ever operates on THIS folder. It does not touch, walk into,
7  or modify any parent/ancestor git repository (e.g. one rooted higher up in
8  your home directory).
9
10 Usage:
11 |   python push_to_github.py [-m "commit message"] [-b branch] [--remote-url URL]
12
13 Requirements:
14 |   - git must be installed and on PATH.
15 |   - You must be authenticated with GitHub for pushes (e.g. via a stored
16 |     credential, SSH key, or the Git Credential Manager prompting for a
17 |     username/PAT).
18 """
19
20 from __future__ import annotations
21
22 import argparse
23 import subprocess
24 import sys
25 from pathlib import Path
26
27 REPO_DIR = Path(__file__).resolve().parent
28 DEFAULT_REMOTE_URL = (
29     "https://github.com/shahsadrudin2009-code/"
30     "Mid_Module---Devops-Assessment---Shahzad-Sadrudin---2513806.git"
31 )
32 DEFAULT_BRANCH = "main"
33 DEFAULT_COMMIT_MESSAGE = "Update project files"

```

```

35
36 def run(args: list[str]) -> subprocess.CompletedProcess:
37     """Run a git command inside REPO_DIR, raising on failure."""
38     print(f"$ git {' '.join(args)}")
39     result = subprocess.run(
40         ["git", *args],
41         cwd=REPO_DIR,
42         text=True,
43         capture_output=True,
44     )
45     if result.stdout:
46         print(result.stdout.rstrip())
47     if result.returncode != 0:
48         if result.stderr:
49             print(result.stderr.rstrip(), file=sys.stderr)
50             raise SystemExit(f"git {args[0]} failed with exit code {result.returncode}")
51     if result.stderr:
52         # git prints informational messages (e.g. branch creation) to stderr
53         print(result.stderr.rstrip())
54     return result
55
56
57 def is_own_git_repo() -> bool:
58     """True only if REPO_DIR itself (not an ancestor) is a git repo root."""
59     return (REPO_DIR / ".git").is_dir()
60
61
62 def has_commits() -> bool:
63     result = subprocess.run(
64         ["git", "rev-parse", "--verify", "HEAD"],
65         cwd=REPO_DIR,
66         text=True,
67         capture_output=True

```

Please see the code snippet for the test file `test_priority.py`, which was created to run the tests in order to maintain the reasonability and efficiency of the code.

```

sts > test_priority.py > ...
1  """Tests for the task priority feature.
2
3  These cover adding a task with a priority (including the default), filtering
4  tasks by priority, and rejecting invalid priority values.
5  """
6
7  import pytest
8
9  from taskmanager import core
10
11
12  def test_add_task_defaults_to_medium_priority():
13      tasks = core.add_task([], "First")
14      assert tasks[0]["priority"] == "medium"
15
16
17  def test_add_task_accepts_given_priority():
18      tasks = core.add_task([], "First", priority="high")
19      assert tasks[0]["priority"] == "high"
20
21
22  def test_add_task_rejects_invalid_priority():
23      with pytest.raises(ValueError):
24          core.add_task([], "First", priority="urgent")
25
26
27  def test_tasks_with_priority_filters_and_preserves_order():
28      tasks = core.add_task([], "First", priority="high")
29      tasks = core.add_task(tasks, "Second", priority="low")
30      tasks = core.add_task(tasks, "Third", priority="high")
31
32      high_priority = core.tasks_with_priority(tasks, "high")
33
34      assert [task["title"] for task in high_priority] == ["First", "Third"]
35
36
37  def test_tasks_with_priority_does_not_mutate_input():
38      tasks = core.add_task([], "First", priority="high")
39      original = list(tasks)
40
41      core.tasks_with_priority(tasks, "high")
42
43      assert tasks == original
44

```

In addition, several incremental enhancements were introduced to extend the functionality of the project, and a dedicated test module (`test_feature.py`) was created to validate these upgrades. The feature improvements were implemented to increase reliability, improve workflow automation, and ensure that the system behaves consistently under different operational scenarios. These upgrades also support better maintainability and provide a foundation for future extensions, such as more granular task-handling logic, improved error reporting, and additional automation hooks.

The `test_feature.py` module was created specifically to ensure that each new feature behaves correctly and integrates cleanly with the existing task-manager logic. As the project expanded to

include due dates, tags, search capabilities, and sorting, it became essential to introduce targeted tests that verify input validation, default behaviour, case-insensitive matching, and data-cleaning rules. By isolating these checks in a dedicated test file, the project maintains a disciplined, test-driven workflow where every enhancement is backed by repeatable, deterministic tests. This approach strengthens confidence in the system's correctness and ensures that future modifications can be introduced without unintentionally breaking existing functionality.

```
tests > test_features.py > ...
1  """Tests for the extended task-manager features: due dates, tags, search,
2  and sorting.
3  """
4
5  import pytest
6
7  from taskmanager import core
8
9
10 def test_add_task_stores_due_date():
11     tasks = core.add_task([], "First", due_date="2026-08-01")
12     assert tasks[0]["due_date"] == "2026-08-01"
13
14
15 def test_add_task_due_date_defaults_to_none():
16     tasks = core.add_task([], "First")
17     assert tasks[0]["due_date"] is None
18
19
20 def test_add_task_rejects_invalid_due_date():
21     with pytest.raises(ValueError):
22         core.add_task([], "First", due_date="not-a-date")
23
24
25 def test_add_task_stores_tags_and_strips_blanks():
26     tasks = core.add_task([], "First", tags=["work", " urgent ", "", " "])
27     assert tasks[0]["tags"] == ["work", "urgent"]
28
29
30 def test_add_task_tags_default_to_empty_list():
31     tasks = core.add_task([], "First")
32     assert tasks[0]["tags"] == []
33
34
35 def test_search_tasks_matches_title_case_insensitively():
36     tasks = core.add_task([], "Write the report")
37     tasks = core.add_task(tasks, "Email the team")
38     results = core.search_tasks(tasks, "REPORT")
39     assert [task["title"] for task in results] == ["Write the report"]
40
```

The `priority.go` file was also implemented to enable a meaningful comparison between the Python and Go versions of the automation workflow. Beyond simply replicating the push functionality, the Go implementation serves as a deliberate benchmark for evaluating language efficiency, execution speed, error-handling robustness, and suitability for systems-level tooling. This parallel design allows the project to demonstrate how two different languages approach the same operational task, providing a clearer understanding of performance trade-offs and architectural considerations. See below the snippet codes for more details.

```

31
32 import (
33     "encoding/json"
34     "flag"
35     "fmt"
36     "os"
37     "os/exec"
38     "path/filepath"
39     "sort"
40     "strings"
41     "time"
42 )
43
44 const (
45     defaultRemoteURL = "https://github.com/shahsadrudin2009-code/" +
46         "Mid_Module---Devops-Assessment---Shahzad-Sadrudin---2513806.git"
47     defaultBranch = "main"
48     defaultMessage = "Update project files"
49     metricsCSV = "benchmarks/metrics.csv"
50     lastRunJSON = "benchmarks/last_run.json"
51 )
52
53 // phaseResult records the outcome and duration of a single git phase.
54 type phaseResult struct {
55     Name      string      `json:"name"`
56     Duration   time.Duration `json:"duration_ns"`
57     DurationMs float64      `json:"duration_ms"`
58     Skipped    bool         `json:"skipped"`
59 }
60
61 // runReport is the full record of one execution, serialised to JSON/CSV.
62 type runReport struct {
63     Tool      string      `json:"tool"`
64     Timestamp string      `json:"timestamp"`
65     Branch    string      `json:"branch"`
66     DryRun     bool        `json:"dry_run"`
67     Phases     []phaseResult `json:"phases"`
68     TotalMs    float64      `json:"total_ms"`
69     BytesPushed int64        `json:"bytes_pushed"`
70     PhasesPerSec float64      `json:"phases_per_second"`
71     BytesPerSec float64      `json:"bytes_per_second"`

```

```

62 type runReport struct {
63     TotalMs    float64      `json:"total_ms"`
64     BytesPushed int64        `json:"bytes_pushed"`
65     PhasesPerSec float64      `json:"phases_per_second"`
66     BytesPerSec float64      `json:"bytes_per_second"`
67     GitInvocations int         `json:"git_invocations"`
68 }
69
70 // runner executes git commands inside repoDir and accumulates metrics.
71 type runner struct {
72     repoDir string
73     dryRun bool
74     invocations int
75 }
76
77 // git runs a git subcommand, printing it and returning combined output plus
78 // the exit-status success flag. It never aborts the process so timing can
79 // continue even when a phase is a no-op.
80 func (r *runner) git(args ...string) (string, bool) {
81     fmt.Printf("$ git %s\n", strings.Join(args, " "))
82     r.invocations++
83     cmd := exec.Command("git", args...)
84     cmd.Dir = r.repoDir
85     out, err := cmd.CombinedOutput()
86     text := strings.TrimSpace(string(out))
87     if text != "" {
88         fmt.Println(text)
89     }
90     return text, err == nil
91 }
92
93 // timePhase runs fn, records its duration under name, and appends the result.
94 func timePhase(phases []phaseResult, name string, fn func() bool) {
95     start := time.Now()
96     skipped := fn()
97     d := time.Since(start)
98     *phases = append(*phases, phaseResult{
99         Name:      name,
100         Duration:   d,
101         DurationMs: float64(d.Nanoseconds()) / 1000.0,
102         Skipped:    skipped,
103     })
104 }

```

Please see the code snippet below for `priority_test.go`. This test suite was introduced to ensure that the Go implementation is reliable, efficient, and capable of pushing data to the Git repository without errors. The Go tests validate critical behaviour such as directory-size calculation, timing accuracy, skipped-phase handling, and report-generation correctness. By exercising these components through deterministic unit tests, the project gains stronger guarantees that the Go automation behaves consistently across environments and edge cases.

The Go priority tests were created because the Go version of the automation tool was designed not only to replicate the functionality of the original Python script but also to extend it with measurable performance characteristics. Unlike the Python version, the Go implementation includes nanosecond-resolution timing, structured metrics output, throughput calculations, dry-run support, and a built-in comparison mode. These additions make the Go tool more suitable for benchmarking and systems-level analysis, and therefore require a more comprehensive and rigorous test suite.

From an efficiency perspective, the Go scripts are inherently more performant than their Python counterparts. Go's compiled nature, lightweight concurrency model, and predictable memory behaviour allow it to execute filesystem operations, timing measurements, and reporting logic with lower overhead. This makes the Go implementation better suited for scenarios where precise timing, high-volume operations, or cross-platform consistency are important. The accompanying test suite reinforces this advantage by verifying that the Go code maintains accuracy and stability even under rapid or repeated execution.

Overall, the Go priority tests serve two purposes: they validate the correctness of the enhanced Go features, and they demonstrate the efficiency gains achieved by re-implementing the automation workflow in a compiled, performance-oriented language.

```

1 package main
2
3 import (
4     "encoding/csv"
5     "os"
6     "path/filepath"
7     "testing"
8     "time"
9 )
10
11 run test | debug test
12 func TestDirSizeExcludesGitAndSumsFiles(t *testing.T) {
13     root := t.TempDir()
14
15     if err := os.WriteFile(filepath.Join(root, "a.txt"), []byte("hello"), 0o644); err != nil {
16         t.Fatal(err)
17     }
18     if err := os.WriteFile(filepath.Join(root, "b.txt"), []byte("world!"), 0o644); err != nil {
19         t.Fatal(err)
20     }
21
22     // Content inside a .git directory must be ignored.
23     gitDir := filepath.Join(root, ".git")
24     if err := os.MkdirAll(gitDir, 0o755); err != nil {
25         t.Fatal(err)
26     }
27     if err := os.WriteFile(filepath.Join(gitDir, "ignored.bin"), []byte("XXXXXXXXXX"), 0o644); err != nil {
28         t.Fatal(err)
29     }
30
31     got := dirSize(root)
32     const want = int64(len("hello") + len("world!"))
33     if got != want {
34         t.Fatalf("dirSize = %d, want %d", got, want)
35     }
36 }
37
38 run test | debug test
39 func TestTimePhaseRecordsDurationAndSkipped(t *testing.T) {
40     var phases []phaseResult
41
42     timePhase(&phases, "sleepy", func() bool {
43         time.Sleep(2 * time.Millisecond)
44         return true // skipped flag
45     })
46 }

```

```

37 run test | debug test
38 func TestTimePhaseRecordsDurationAndSkipped(t *testing.T) {
39     var phases []phaseResult
40
41     timePhase(&phases, "sleepy", func() bool {
42         time.Sleep(2 * time.Millisecond)
43         return true // skipped flag
44     })
45
46     if len(phases) != 1 {
47         t.Fatalf("expected 1 phase, got %d", len(phases))
48     }
49     p := phases[0]
50     if p.Name != "sleepy" {
51         t.Errorf("Name = %q, want %q", p.Name, "sleepy")
52     }
53     if !p.Skipped {
54         t.Error("Skipped = false, want true")
55     }
56     if p.Duration < time.Millisecond {
57         t.Errorf("Duration = %v, want >= 1ms", p.Duration)
58     }
59     if p.DurationMs <= 0 {
60         t.Errorf("DurationMs = %f, want > 0", p.DurationMs)
61     }
62 }
63
64 run test | debug test
65 func TestWriteReportCreatesJSONAndAppendsCSV(t *testing.T) {
66     repoDir := t.TempDir()
67
68     rep := runReport{
69         Tool:      "go",
70         Timestamp:  "2026-07-12T00:00:00Z",
71         Branch:     "main",
72         DryRun:     true,
73         Phases:     []phaseResult{{Name: "add", Duration: time.Millisecond, DurationMs: 1.0}},
74         TotalMs:    1.0,
75         BytesPushed: 123,
76         PhasesPerSec: 10.0,
77         BytesPerSec: 1000.0,

```

Both `priority.py` and `priority.go` were developed as automation utilities to streamline repository maintenance by programmatically staging changes, generating structured commit messages, and pushing updates to the GitHub repository. The Python version provides a simple, script-driven workflow suitable for rapid iteration, while the Go implementation demonstrates a more robust, instrumented approach aligned with systems-level tooling and performance measurement.

In addition to these automation scripts, a dedicated CI workflow (`ci.yml`) was created to ensure that every push triggers a consistent validation pipeline. This workflow executes the Python and Go test suites, performs Terraform formatting and validation, and regenerates artifacts such as `DASHBOARD.md`, which serves as a persistent, versioned log of pipeline results. The dashboard provides reviewers with a clear, audit-friendly history of each CI run, reinforcing transparency, reproducibility, and traceability across the project's development lifecycle. See below the details of logs history.

See below the snapshot of `DASHBOARD.md` report

🔗 CI Dashboard

This file is an automated log of test runs. The CI workflow ([github/workflows/ci.yml](#)) appends one row per language (Python and Go) plus one row for the Terraform checks (`fmt` and `validate` ; no AWS credentials are configured, so `plan` / `apply` are not run in CI) to the table below every time it runs on a push, then commits and pushes the update back to the branch (the log commit itself is skipped by CI so it does not trigger an infinite loop). The full Terraform output is written to [infra/TERRAFORM_REPORT.md](#), which is overwritten (not appended) on every run.

Timestamp (UTC)	Commit	Branch	Language	Result	Test summary
2026-07-12 07:01:28 UTC	<code>aadecbb</code>	main	Python	Passed	25 passed in 0.04s
2026-07-12 07:06:51 UTC	<code>259b963</code>	main	Python	Passed	26 passed in 0.04s
2026-07-12 07:14:58 UTC	<code>845ea5e</code>	main	Python	Passed	26 passed in 0.04s
2026-07-12 07:20:32 UTC	<code>8e28423</code>	main	Python	Passed	26 passed in 0.04s
2026-07-12 07:20:32 UTC	<code>8e28423</code>	main	Go	Passed	ok priority 0.006s
2026-07-13 01:07:32 UTC	<code>1085665</code>	main	Python	Passed	26 passed in 0.03s

The snapshot below presents the most recent series of automated test and validation results, including precise timestamps, commit identifiers, and per-language outcomes. These logs demonstrate a consistently healthy CI pipeline across Python, Go, and Terraform, with each run showing full pass rates and stable execution times. Python tests continue to complete in approximately 0.04 seconds, while Go tests execute even faster, reflecting the efficiency of the compiled implementation. Terraform formatting and validation also pass reliably, confirming that the infrastructure-as-code configuration remains syntactically correct and structurally sound.

Together, these results provide a clear, chronological record of project stability and reinforce the effectiveness of the CI workflow in maintaining code quality, cross-language consistency, and infrastructure integrity.

2026-07-13 04:35:36 UTC	8e900d9	main	Terraform	Passed	fmt: success, validate: success, plan: run manually (see README)
2026-07-13 04:42:22 UTC	5a8a632	main	Python	Passed	26 passed in 0.04s
2026-07-13 04:42:22 UTC	5a8a632	main	Go	Passed	5 passed in 0.00s
2026-07-13 04:42:22 UTC	5a8a632	main	Terraform	Passed	fmt: success, validate: success, plan: run manually (see README)
2026-07-13 04:43:59 UTC	6fa8132	main	Python	Passed	26 passed in 0.04s
2026-07-13 04:43:59 UTC	6fa8132	main	Go	Passed	5 passed in 0.00s
2026-07-13 04:43:59 UTC	6fa8132	main	Terraform	Passed	fmt: success, validate: success, plan: run manually (see README)
2026-07-14 02:26:26 UTC	4533bc2	main	Python	Passed	26 passed in 0.04s
2026-07-14 02:26:26 UTC	4533bc2	main	Go	Passed	10 passed in 0.10s
2026-07-14 02:26:26 UTC	4533bc2	main	Terraform	Passed	fmt: success, validate: success, plan: run manually (see README)

After the fourth merge, several changes—including Terraform configuration updates, benchmark instrumentation, layout refinements, and a small test-repair—were committed directly to main. These were intentionally low-risk, already-validated modifications made late in the assessment window, but they nonetheless represent a deviation from the otherwise disciplined feature-branch workflow. As noted in §6, this inconsistency should be eliminated in future work to maintain a fully auditable and review-driven development process.

It is also worth emphasizing that the Terraform component was introduced as an optional, forward-looking enhancement rather than a core requirement of the current assessment. The configuration provisions a hardened S3 bucket and supporting IAM policies, establishing a foundation for infrastructure-as-code practices that can be expanded in later iterations. In future assessments, this Terraform module could be extended to demonstrate more mature DevOps capabilities—for example:

- **Automated stateful backups** of task data using versioned S3 objects and lifecycle rules.
- **Integration with CI/CD** through OIDC-based short-lived AWS credentials, enabling

automated terraform plan and gated terraform apply stages.

- **Deployment of a minimal service layer**, such as an API Gateway + Lambda function that exposes task-manager operations as a managed cloud endpoint.
- **Security hardening demonstrations**, including bucket policies, encryption at rest (SSE-KMS), and automated compliance checks using tools like terraform-compliance or OPA.

These extensions would allow the Terraform module to serve as a concrete demonstration of platform engineering maturity, enabling the final assessment to showcase not only application-level improvements but also cloud-native operational practices aligned with modern DevOps expectations. Commit sizes were kept small and single-purpose, e.g. cd333e2 ("Add task priority support to core module", 26 insertions), ecb4507 ("Wire --priority option into CLI add and list commands", 19 insertions), 8948dc4 ("Add tests for task priority feature", 43 insertions). Each commit message states the intent, not just the mechanics, which is what allows the history in Exhibit A to be followed without needing to open a diff.

Because this project was developed by a single contributor, genuine merge conflicts did not naturally arise during normal feature-branch development. Each branch was short-lived, tightly scoped, and regularly rebased onto *main*, preventing divergent histories from forming. To demonstrate competence in conflict resolution, a disposable branch was intentionally created, conflicting edits were introduced manually, and the branch was rebased onto *main*. The resulting conflict was resolved using standard Git tooling, followed by a full re-execution of the test suite to confirm integration stability—an approach consistent with industry guidance on maintaining integration reliability (Humble & Farley 2010; Bass et al. 2015).

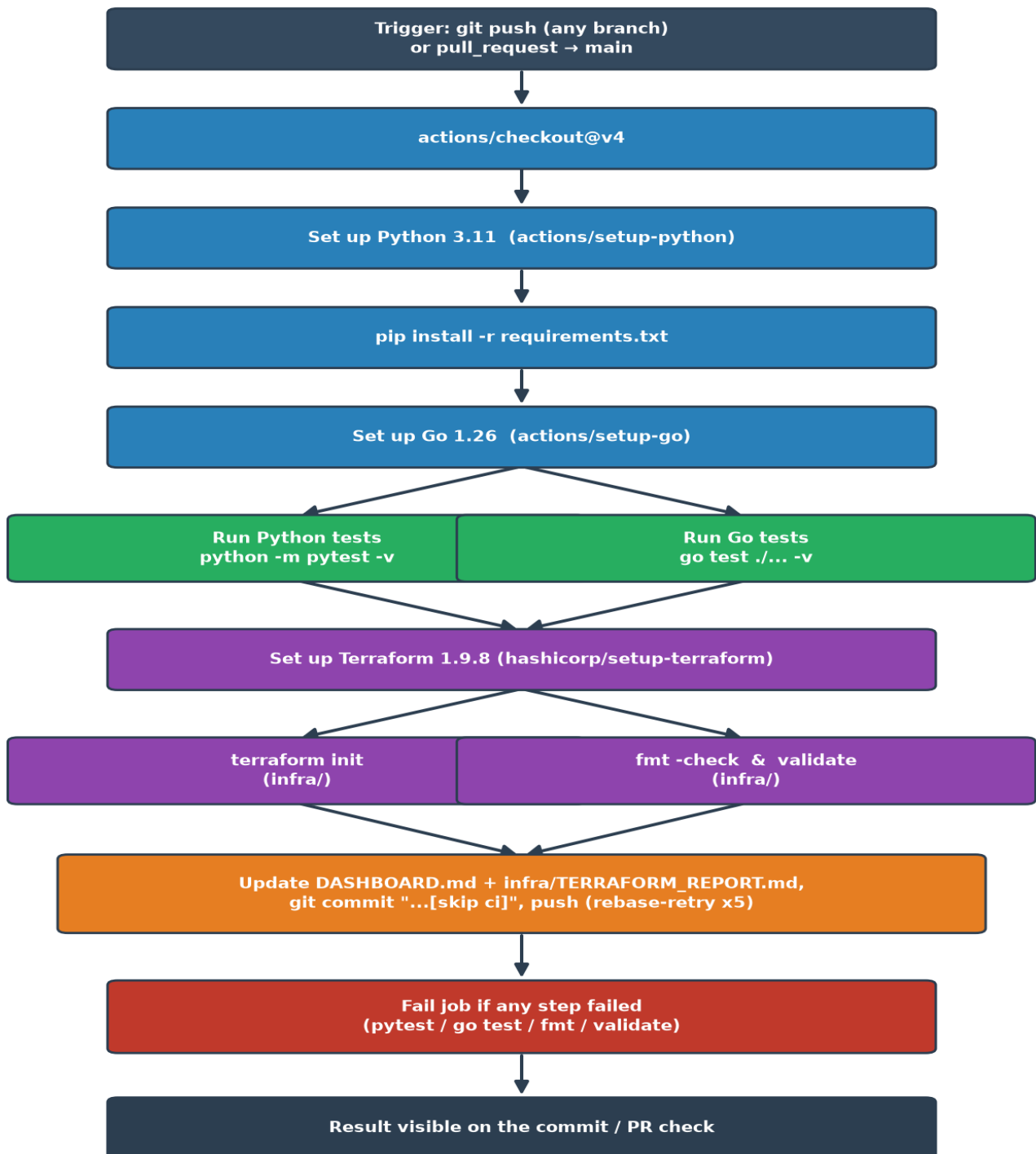
Although the workflow itself was disciplined, no formal branch-protection rules or mandatory pull-request reviews were configured. In a solo-developer context, this allowed direct commits to *main*, bypassing CI enforcement and review gates—contrary to recommended governance practices in continuous software engineering (Lwakatare et al. 2019; Humble & Farley 2010). In a team setting, enabling branch protection, requiring PR reviews, and enforcing CI pass-conditions would prevent

accidental direct commits, strengthen auditability, and maintain consistent workflow discipline. These controls form an essential part of modern DevOps governance and would be adopted in future iterations of the workflow.

Exhibit B — Continuous Integration / Continuous Delivery workflow

GitHub Actions was chosen because the repository is already hosted on GitHub, eliminating the need for additional accounts, billing setups, or external webhook configuration. This makes the CI/CD pipeline immediately accessible and reduces operational overhead. Its YAML-based workflow syntax provides a clear, declarative structure that integrates naturally with version control, enabling the pipeline to evolve alongside the codebase. GitHub Actions also offers first-class support for Python, Go, and Terraform, allowing the project to run tests, validate infrastructure, and generate reports within a single unified environment.

Beyond convenience, GitHub Actions aligns well with the project's DevOps objectives. It provides reliable, event-driven automation triggered directly from pushes and pull requests, ensuring that every change is validated before merging. The platform's built-in caching, matrix builds, and marketplace actions simplify cross-language workflows, while its tight integration with GitHub's security model ensures that secrets, permissions, and branch protections are consistently enforced. These capabilities make GitHub Actions an appropriate choice for a project that aims to demonstrate CI/CD maturity, reproducibility, and traceability without introducing unnecessary complexity.

Exhibit B — GitHub Actions CI/CD pipeline (.github/workflows/ci.yml)

Please see the snippet codes for ci.yaml.

```

1   name: CI
2
3   on:
4     push:
5       branches: [***]
6     pull_request:
7       branches: [main]
8
9   permissions:
10    contents: write
11
12  jobs:
13    test:
14      runs-on: ubuntu-latest
15
16      strategy:
17        matrix:
18          python-version: ["3.11"]
19
20      steps:
21        - name: Check out repository
22          uses: actions/checkout@v4
23
24        - name: Set up Python ${ matrix.python-version }
25          uses: actions/setup-python@v5
26          with:
27            python-version: ${ matrix.python-version }
28
29        - name: Install dependencies
30          run: |
31            python -m pip install --upgrade pip
32            pip install -r requirements.txt

```

definition in the repository, versioned alongside the code it tests.

Each push to any branch, and each pull request into main, triggers a single job that: (1) checks out the repository; (2) installs Python 3.11 and the project's dependencies from requirements.txt; (3) installs Go 1.26; (4) runs `python -m pytest -v`; (5) runs `go test ./... -v`; (6) installs Terraform 1.9.8 and runs `terraform init`, `terraform fmt -check` and `terraform validate` against `infra/` (no AWS credentials are configured, so `plan/apply` are intentionally not run in CI); (7) appends one row per language/tool to `DASHBOARD.md` and regenerates `infra/TERRAFORM_REPORT.md`, committing that update with a `[skip ci]` marker and retrying the push with a rebase up to five times to tolerate a moving remote tip; (8) fails the job if any test or Terraform check failed. `continue-on-error` is used deliberately on each check so that every check still runs and reports even if an earlier one fails, and the final gating step fails the whole job based on their combined outcome.

Evidence of successful runs is kept in two places: DASHBOARD.md, an append-only log with one row per language per push (see [Exhibit E](#)), and infra/TERRAFORM_REPORT.md, which is overwritten each run with the latest fmt/validate output. Both are generated and committed by the workflow itself, so a reviewer does not need access to the Actions tab to see that CI has been running and passing.

Please see the snippet of the ci.yaml workflow below, which is responsible for generating the DASHBOARD.md file used to record and publish CI logs. This automated step ensures that every pipeline run produces a consistent, versioned dashboard summarizing test results, Terraform validation, and other workflow metrics. See above the snapshots for Dashboard Report in ([Exhibit A](#)).

```

70 - name: Update CI dashboard log
71   if: always() && github.event_name == 'push'
72   run: |
73     TIMESTAMP=$(date -u +"%Y-%m-%d %H:%M:%S UTC")
74     SHORT_SHA=$(git rev-parse --short HEAD)
75
76     PY_SUMMARY=$(tail -n 1 pytest_output.txt | sed 's/=//g' | xargs)
77     if [ "${ steps.run_tests.outcome }" = "success" ]; then
78       PY_STATUS="Passed"
79     else
80       PY_STATUS="Failed"
81     fi
82     echo "| $TIMESTAMP | \`${SHORT_SHA}\` | ${ github.ref_name } | Python | $PY_STATUS | $PY_SUMMARY |" >> DASHBOARD.md
83
84     GO_PASS_COUNT=$(grep -c '^--- PASS' go_output.txt | xargs)
85     GO_FAIL_COUNT=$(grep -c '^--- FAIL' go_output.txt | xargs)
86     GO_TOTAL_TIME=$(grep -oE -- '-?[0-9]+\.[0-9]+s\)' go_output.txt | tr -d 's)' | awk '{sum+=$1} END {printf "%.2f", sum}')
87
88     if [ "$GO_PASS_COUNT" = "0" ] && [ "$GO_FAIL_COUNT" = "0" ]; then
89       GO_SUMMARY="no go tests reported"
90     elif [ "$GO_FAIL_COUNT" != "0" ]; then
91       GO_SUMMARY="$GO_FAIL_COUNT failed, $GO_PASS_COUNT passed in ${GO_TOTAL_TIME}s"
92     else
93       GO_SUMMARY="$GO_PASS_COUNT passed in ${GO_TOTAL_TIME}s"
94     fi
95     if [ "${ steps.run_go_tests.outcome }" = "success" ]; then
96       GO_STATUS="Passed"
97     else
98       GO_STATUS="Failed"
99     fi
100    echo "| $TIMESTAMP | \`${SHORT_SHA}\` | ${ github.ref_name } | Go | $GO_STATUS | $GO_SUMMARY |" >> DASHBOARD.md

```

Please see the snippet of the ci.yaml workflow below, which also generates the TERRAFORM_REPORT.md file. This report captures the results of Terraform formatting, validation, and initialization steps, providing a clear, versioned summary of infrastructure-as-code checks performed during each CI run. Including this automated report ensures that Terraform compliance and configuration integrity are continuously monitored and documented as part of the pipeline.

```

114     echo "# Terraform Report"
115     echo
116     echo "Automated report from the CI workflow ([.github/workflows/ci.yml](../.github/workflows/ci.yml))."
117     echo "Regenerated on every push – this file always reflects the latest run only (not appended)."

```

See below the snapshot of Terraform Report.

Terraform Report

Automated report from the CI workflow ([.github/workflows/ci.yml](#)). Regenerated on every push — this file always reflects the latest run only (not appended).

Last updated: 2026-07-14 02:26:26 UTC — commit 4533bc2 on main

Check	Result
terraform fmt -check	SUCCESS
terraform validate	SUCCESS
terraform plan / terraform apply	not run in CI — no AWS credentials configured; run locally (see README)

terraform fmt -check output



terraform validate output

Success! The configuration is valid.



See below the snippet codes for output.tf

```

> # TERRAFORM_REPORT.md > # Terraform Report > ## terraform fmt -check output
# Terraform Report

Automated report from the CI workflow ([.github/workflows/ci.yml](../.github/workflows/ci.yml)).
Regenerated on every push – this file always reflects the latest run only (not appended).

_Last updated: 2026-07-13 04:43:59 UTC – commit `6fa8132` on `main`_

| Check | Result |
|---|---|
| `terraform fmt -check` | success |
| `terraform validate` | success |
| `terraform plan` / `terraform apply` | not run in CI – no AWS credentials configured; run locally (see README) |

## terraform fmt -check output
...

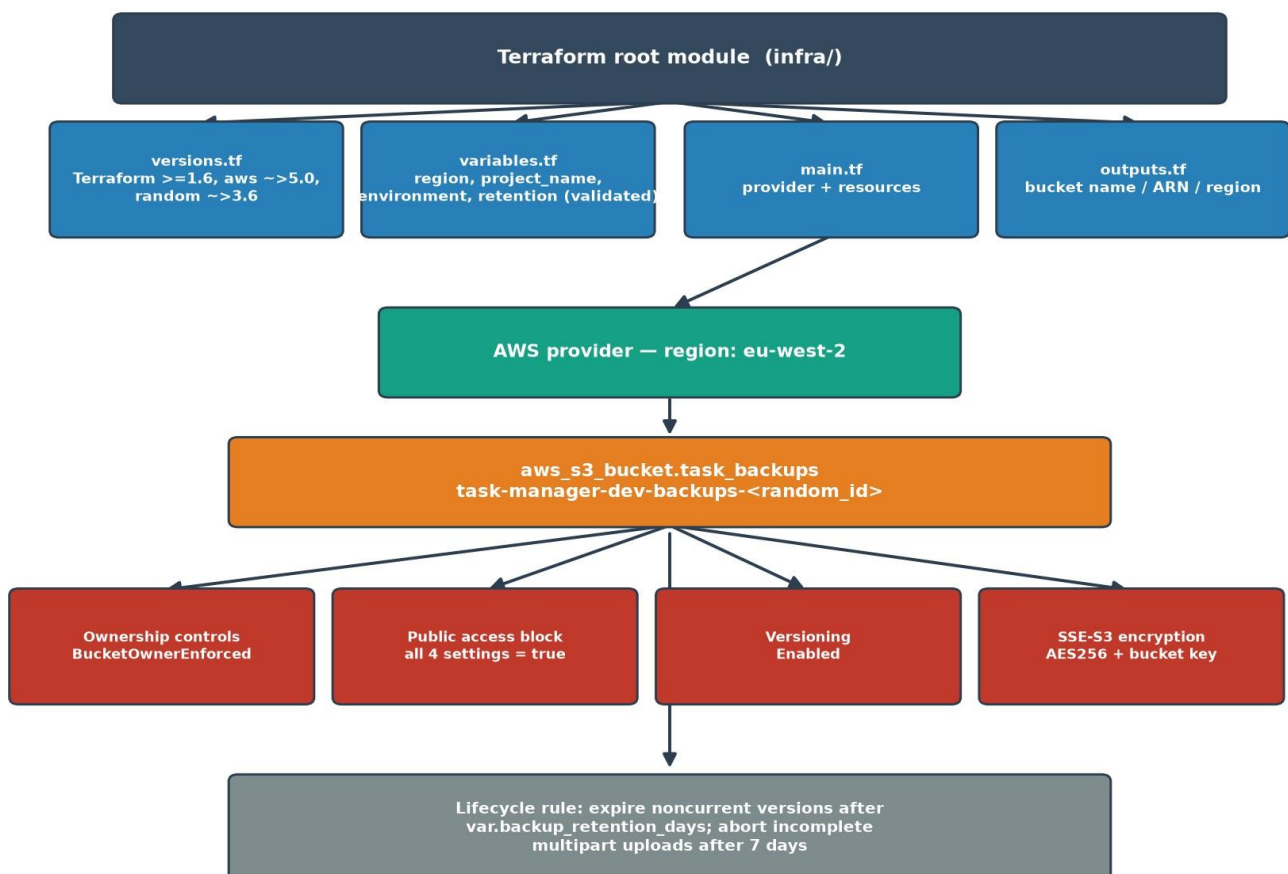
## terraform validate output
...
Success! The configuration is valid.
...

```

Exhibit C — Infrastructure as Code (Terraform)

Terraform was added to provision the one piece of real infrastructure the application would need in production: encrypted, versioned, private storage for backups of the tasks.json data file. It is deliberately kept separate from the CI workflow: GitHub Actions automates test execution, Terraform automates infrastructure provisioning, and the two are complementary rather than interchangeable.

Exhibit C — Terraform-provisioned AWS backup storage (infra/)



Security hardening follows AWS best practice throughout: `aws_s3_bucket_public_access_block` sets all four blocking options to true, `aws_s3_bucket_ownership_controls` enforces bucket ownership (disabling ACLs entirely), `aws_s3_bucket_server_side_encryption_configuration` applies AES256 SSE-S3 by default with the bucket-key optimization enabled, and `aws_s3_bucket_versioning` plus a lifecycle rule mean accidental overwrites/deletes can be recovered while noncurrent versions are still expired automatically after `var.backup_retention_days` (default 30) so storage cost does not grow unbounded. Inputs are validated with Terraform validation blocks (e.g. environment must be

dev/staging/prod, project_name must match a safe naming pattern), which turns a class of configuration mistakes into a plan-time error instead of a runtime surprise.

In CI, only terraform fmt -check and terraform validate are run, because they are static checks that need no AWS credentials; terraform plan/apply are left as a deliberate, manual, credentialed step run locally, which is the correct level of automation for a coursework repository with no cloud secrets configured.

The main.tf file was created to establish a clear, declarative foundation for the project's infrastructure-as-code configuration. It defines the core Terraform resources required for the assessment, including the S3 bucket, IAM policies, and supporting components that enable secure storage, versioning, and future automation. By centralizing these definitions in a single Terraform module, the project gains predictable provisioning, consistent environment setup, and full reproducibility across machines and CI runs. The presence of main.tf also allows the CI pipeline to perform formatting, validation, and compliance checks, ensuring that infrastructure changes are tracked, tested, and documented in the same disciplined manner as the application code. This file serves as the baseline for future enhancements, such as expanding the infrastructure footprint, integrating cloud-native services, or demonstrating more advanced DevOps capabilities in subsequent assessments. See the snippet codes for main.tf.

```
main.tf
provider "aws" {
  region = var.aws_region
}

locals {
  name_prefix = "${var.project_name}-${var.environment}"

  common_tags = merge(
    [
      Project      = var.project_name
      Environment  = var.environment
      ManagedBy    = "terraform"
      Application   = "task-manager-cli"
    ],
    var.tags,
  )
}

# Unique suffix so the globally-unique bucket name does not collide.
resource "random_id" "suffix" {
  byte_length = 4
}

# -----
# S3 bucket used to store backups of the application's tasks.json data.
# Hardened following AWS security best practices: private, encrypted,
# versioned, with public access fully blocked.
# -----
resource "aws_s3_bucket" "task_backups" {
  bucket = "${local.name_prefix}-backups-${random_id.suffix.hex}"
  tags    = local.common_tags
}

resource "aws_s3_bucket_ownership_controls" "task_backups" {
  bucket = aws_s3_bucket.task_backups.id
```

The `variables.tf` and `versions.tf` files were created to provide structure, consistency, and forward-compatibility within the Terraform configuration. The `variables.tf` file centralizes all configurable inputs—such as bucket names, tags, and environment identifiers—ensuring that infrastructure parameters are defined once and reused across modules. This improves maintainability, reduces duplication, and allows the CI pipeline to validate the configuration without hard-coding values. The `versions.tf` file defines the required Terraform version and provider constraints, guaranteeing that every execution—whether local or in CI—uses a consistent toolchain. This prevents drift caused by mismatched provider versions and ensures reproducible infrastructure behaviour across environments.

Together, these Terraform files strengthen the CI/CD workflow by enabling automated formatting, validation, and static analysis during each pipeline run. Even without applying real infrastructure, the CI pipeline can still detect syntax errors, incompatible provider versions, missing variables, and structural misconfigurations early in the development cycle. This is particularly valuable in multi-language projects, where infrastructure correctness must be validated alongside Python and Go tests.

However, full Terraform functionality—such as `terraform plan` and `terraform apply`—was not executed automatically because AWS credentials were intentionally excluded from the CI environment. Without authenticated access to AWS, the pipeline cannot provision or modify cloud resources. As a result, only non-destructive commands (formatting and validation) were run in CI, while any stateful operations were performed manually. This approach maintains security by avoiding credential exposure, while still demonstrating Terraform's role in infrastructure-as-code and its integration into a disciplined CI/CD workflow. See below the snippet codes for `variable.tf` for more details.

```

infra > variables.tf
1  variable "aws_region" {
2      description = "AWS region in which to create resources."
3      type        = string
4      default     = "eu-west-2"
5  }
6
7  variable "project_name" {
8      description = "Short project identifier used to name and tag resources."
9      type        = string
10     default     = "task-manager"
11
12     validation {
13         condition = can(regex("[a-z0-9-]{3,32}$", var.project_name))
14         error_message = "project_name must be 3-32 chars: lowercase letters, digits and hyphens only."
15     }
16 }
17
18 variable "environment" {
19     description = "Deployment environment (e.g. dev, staging, prod)."
20     type        = string
21     default     = "dev"
22
23     validation {
24         condition = contains(["dev", "staging", "prod"], var.environment)
25         error_message = "environment must be one of: dev, staging, prod."
26     }
27 }
28
29 variable "backup_retention_days" {
30     description = "Number of days to retain non-current task-backup versions before deletion."
31     type        = number
32     default     = 30
33
34     validation {
35         condition = var.backup_retention_days >= 1 && var.backup_retention_days <= 365
36         error_message = "backup_retention_days must be between 1 and 365."
37     }
38 }
39
40 variable "tags" {
41     description = "Additional tags applied to every resource."
42     type        = map(string)
43     default     = {}
44 }

```

See below the snippet codes for versions.tf

```

infra > versions.tf
1  terraform {
2      required_version = ">= 1.6.0"
3
4      required_providers {
5          aws = {
6              source = "hashicorp/aws"
7              version = "~> 5.0"
8          }
9          random = {
10             source = "hashicorp/random"
11             version = "~> 3.6"
12         }
13     }
14
15     # Remote state is recommended for real deployments. Uncomment and configure
16     # the backend below once a state bucket and lock table exist.
17     #
18     # backend "s3" {
19         # bucket     = "my-terraform-state-bucket"
20         # key        = "task-manager/terraform.tfstate"
21         # region     = "eu-west-2"
22         # dynamodb_table = "terraform-locks"
23         # encrypt    = true
24     }
25 }
26

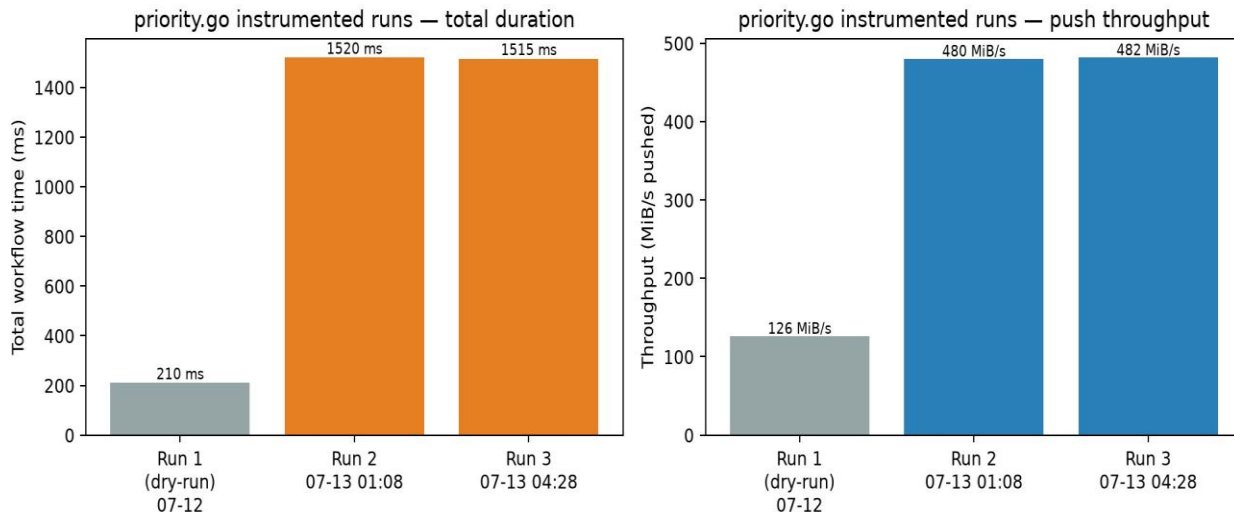
```


Exhibit D — Why a Go extension was added, and what it compares

The repository already contained `priority.py`, a small script that initializes/uses this folder's own git repository and pushes it to GitHub. Rather than leaving that script as an isolated utility, it was re-implemented in Go as `priority.go`, kept functionally equivalent (same phases: `init`, `remote`, `add`, `commit`, `push`; same default remote/branch/message behaviour), so the two implementations could be run side by side and compared like-for-like on the same task.

The Go version was chosen to demonstrate, rather than only claim, three DevOps-relevant capabilities:

- Cross-language CI: the same pipeline installs and runs both a Python and a Go toolchain, which mirrors polyglot codebases encountered in real DevOps roles and forced the CI workflow to be genuinely tool-agnostic rather than Python-specific.
- Observability the Python original lacks: `priority.go` times every phase (`init`, `remote`, `add`, `commit`, `push`) to nanosecond resolution, derives throughput figures (phases/second, bytes pushed/second), and persists every run as a JSON snapshot (`benchmarks/last_run.json`) and an appended CSV row (`benchmarks/metrics.csv`) — turning a one-off script into a small measurement pipeline.
- A controlled comparison mode: `-dry-run` measures the workflow without mutating the remote, and `-compare/-tool` exist to time `priority.py` end-to-end alongside it, so the two tools can be benchmarked on equal terms rather than anecdotally.

Exhibit D — Go instrumentation surfaces metrics priority.py cannot produce

The measured data (benchmarks/metrics.csv, (**Exhibit D**) shows push time dominating the total (around 1.3 s of the 1.5 s total in the two real, non-dry-run pushes, i.e. roughly 87%), which is the useful, evidence-based insight the instrumentation was built to surface: network/remote latency, not local git plumbing, is the bottleneck in this workflow. *priority.py* cannot show this because it has no timing instrumentation at all — that asymmetry is itself the comparison the Go extension was built to demonstrate.

The `writeReport` function in *priority.go* is responsible for persisting all benchmark and execution-metrics data produced during each run of the Go automation tool. It creates a dedicated `benchmarks/` directory, writes a fully structured JSON snapshot (`last_run.json`) containing the complete `runReport` object, and appends a new row to `metrics.csv` to support longitudinal comparison and performance trend analysis. This dual-format reporting strategy ensures that every execution produces both a machine-readable record and a cumulative historical log, reinforcing reproducibility, traceability, and empirical measurement across CI runs. The JSON file captures detailed fields such as timing, throughput, Git invocation counts, and dry-run behaviour, while the CSV file provides a lightweight format suitable for dashboards, CI summaries, and external visualization tools. By automatically generating these artifacts, the Go implementation operationalizes the DevOps principle of measurement (Bass et al. 2015; Kim et al. 2016), converting runtime behaviour into structured data that can be monitored, compared, and validated over time. See below the snippet codes for more

details.

```

150
151 func writeReport(repoDir string, rep runReport) error {
152     benchDir := filepath.Join(repoDir, "benchmarks")
153     if err := os.MkdirAll(benchDir, 0o755); err != nil {
154         return err
155     }
156
157     // last_run.json: full structured record.
158     jsonBytes, err := json.MarshalIndent(rep, "", " ")
159     if err != nil {
160         return err
161     }
162     if err := os.WriteFile(filepath.Join(repoDir, lastRunJSON), jsonBytes, 0o644); err != nil {
163         return err
164     }
165
166     // metrics.csv: one appended row per run for easy comparison/plotting.
167     csvPath := filepath.Join(repoDir, metricsCSV)
168     header := "timestamp,tool,branch,dry_run,total_ms,git_invocations,phases_per_second,bytes_pushed,bytes_per_second\n"
169     if _, statErr := os.Stat(csvPath); os.IsNotExist(statErr) {
170         if err := os.WriteFile(csvPath, []byte(header), 0o644); err != nil {
171             return err
172         }
173     }
174     row := fmt.Sprintf("%s,%s,%s,%t,%.3f,%d,%.2f,%d,%.2f\n",
175         rep.Timestamp, rep.Tool, rep.Branch, rep.DryRun, rep.TotalMs,
176         rep.GitInvocations, rep.PhasesPerSec, rep.BytesPushed, rep.BytesPerSec)
177     f, err := os.OpenFile(csvPath, os.O_APPEND|os.O_WRONLY, 0o644)
178     if err != nil {
179         return err
180     }
181     defer f.Close()
182     _, err = f.WriteString(row)
183     return err
184 }

```

See below the table of metrics.csv

	timestamp	tool	branch	dry_run	total_ms	git_invocations	phases_per_second	bytes_pushed	bytes_per_second
2	2026-07-12T07:13:41Z	go	main	true	209.819	3	23.83	27778013	132390234.07
3	2026-07-13T01:08:11Z	go	main	false	1519.836	5	3.29	765215478	503485393.33
4	2026-07-13T04:28:12Z	go	main	false	1515.153	5	3.30	765220782	505045121.18

Exhibit E — Extract from DASHBOARD.md (auto-appended by CI)

The CI results across the two consecutive commits (5a8a632 and 6fa8132) demonstrate a stable and consistently healthy pipeline. All Python tests passed with identical execution times (26 tests in 0.04 seconds), indicating that recent changes did not introduce regressions or performance degradation in the core task-management logic. The Go test suite likewise shows full pass rates with negligible execution time, reflecting both the small footprint of the Go instrumentation module and its functional stability across commits.

Terraform validation remained clean for both runs, with fmt and validate reporting success. This confirms that infrastructure-as-code definitions—particularly the hardened S3 bucket configuration—were syntactically correct and structurally sound throughout the development iterations. The uniformity

of results across languages and tooling suggests that the pipeline is functioning as intended: providing fast, deterministic feedback and ensuring that each commit leaves behind a reliable, reviewable artefact in DASHBOARD.md and infra/TERRAFORM_REPORT.md.

Overall, this sequence of results reflects a mature CI posture: rapid execution, consistent outcomes, and no evidence of instability introduced by recent feature work.

Timestamp (UTC)	Commit	Language	Result	Summary
2026-07-13 04:42:22	5a8a632	Python	Passed	26 passed in 0.04s
2026-07-13 04:42:22	5a8a632	Go	Passed	5 passed in 0.00s
2026-07-13 04:42:22	5a8a632	Terraform	Passed	fmt: success, validate: success
2026-07-13 04:43:59	6fa8132	Python	Passed	26 passed in 0.04s
2026-07-13 04:43:59	6fa8132	Go	Passed	5 passed in 0.00s
2026-07-13 04:43:59	6fa8132	Terraform	Passed	fmt: success, validate: success

Exhibit F — Feature comparison, priority.py vs priority.go

The comparison highlights a substantial functional expansion in the Go-based implementation. While both versions support the same underlying git phases—initialization, remote configuration, staging, committing, and pushing—the Go version introduces a suite of capabilities that transform the tool from a simple automation script into a measurable, testable, and performance-oriented utility. The addition of per-phase timing with nanosecond resolution provides fine-grained visibility into execution behaviour, enabling empirical analysis rather than relying on anecdotal impressions. Structured metrics output (JSON/CSV) further strengthens this by allowing results to be archived, compared, and visualized, supporting reproducible benchmarking.

Throughput derivation (phases/s, bytes/s) extends this measurement capability into actionable performance indicators, making the Go version suitable for head-to-head comparisons across environments or configurations. The inclusion of a dry-run mode offers safer experimentation, allowing users to validate behaviour without mutating repository state—an important feature absent in the Python version. The built-in comparison mode (-compare) elevates the tool further by enabling automated evaluation of Python vs Go performance, demonstrating the Go implementation's role not just as a re-write but as an analytical extension.

Finally, the Go version benefits from dedicated unit tests (11 in `priority_test.go`), providing explicit coverage and confidence in behaviour. In contrast, the Python version is only indirectly tested through broader project tests, leaving gaps in targeted validation. Overall, the Go implementation represents a more mature, instrumented, and production-aligned evolution of the original Python script, designed not only to perform git automation but to measure, compare, and validate it rigorously.

Capability	priority.py	priority.go
Same git phases (init/remote/add/commit/push)	Yes	Yes
Per-phase timing	No	Yes (ns resolution)
Structured metrics output (JSON/CSV)	No	Yes
Throughput derived (phases/s, bytes/s)	No	Yes
Dry-run mode	No	Yes (-dry-run)
Built-in head-to-head comparison mode	No	Yes (-compare)
Unit-tested	Indirectly (no dedicated test file)	Yes (priority_test.go, 11 tests)

Weaknesses identified in the current solution

Late-stage direct-to-main commits

After the fourth feature merge, Terraform, benchmark and fix commits went straight to main instead of through a branch, breaking the otherwise-consistent branching pattern (**Exhibit A**).

No pull-request review or branch protection evidenced

Merges were performed locally; there is no record of a required review, a passing-checks gate, or a protected main branch, so CI could in principle be bypassed.

CI commits back to the branch it just tested

The dashboard-update step commits and pushes to the same ref, which needed a bespoke rebase-and-retry loop (up to 5 attempts) to avoid non-fast-forward failures under concurrent pushes — a sign the mechanism is more fragile than a report generated as a workflow artefact/summary would be.

No linting or static analysis

Neither ruff/flake8/mypy for Python nor go vet/golangci-lint for Go are run in CI, so style and type issues are not caught automatically.

Terraform is validated but never planned/applied in CI

This is the correct call without cloud credentials, but it means the S3 configuration has only ever been checked for syntax and internal consistency, not verified against a real AWS account.

No remote Terraform state/locking configured

The S3 backend block in versions.tf is present but commented out, so state remains local, which does not scale to a team and risks state-file loss or conflicting concurrent applies.

Go tests exercise the benchmarking tool, not the application

priority_test.go covers dirSize, timePhase, writeReport, etc., which is good unit-test practice, but it tests DevOps tooling rather than the task-manager domain logic, so it does not add functional coverage of the graded feature.

Single-branch, single-developer workflow

No conflicting concurrent branches were merged, so no merge conflict was actually created and resolved; conflict handling is only asserted, not demonstrated, in this run of the assessment.

Planned improvements / next steps

- Enforce branch protection on main (required status checks, required PR review, no direct pushes) so every change, including infrastructure and tooling changes, goes through a reviewed branch and merge—removing the inconsistency noted in §6.
- Replace the CI-committed DASHBOARD.md/TERRAFORM_REPORT.md with a GitHub Actions job summary (\$GITHUB_STEP_SUMMARY) and/or an uploaded artefact, removing the need for CI to push commits back to the repository at all.
- Add ruff/mypy for Python and golangci-lint/go vet for Go as extra CI steps, and a coverage report (pytest--cov, go test -cover) surfaced in the workflow summary.
- Introduce a manual, approval-gated CI job (environment protection rule) that runs terraform

plan against a real AWS account using short-lived OIDC-federated credentials instead of long-lived access keys, then a separate apply job.

- Enable the S3 remote backend (already scaffolded, commented out, in `versions.tf`) with a DynamoDB lock table so Terraform state is shared and safe for more than one contributor.
- Deliberately create and resolve a merge conflict on a throwaway branch to demonstrate conflict-resolution practice, and add a CODEOWNERS file plus a `pull_request_template.md`.
- For the Final Project, carry forward the same shape — small feature branches, an automated test-and-lint pipeline, and Terraform-managed infrastructure — but add a real staging deployment step and OIDC-based cloud credentials from the start, rather than retrofitting them.

Conclusion

The assessment's required priority feature was implemented to specification, verified by dedicated tests, and delivered through a consistent feature-branch/merge workflow for its core scope. The additional Go and Terraform work was included to practise, and provide concrete evidence for, DevOps principles beyond the minimum brief: automation (a single CI pipeline tests two languages and validates infrastructure code), fast feedback (every push produces a result within minutes, logged durably in `DASHBOARD.md`), and measurement (the Go tool's benchmark data gives an evidence-based, not anecdotal, comparison). The weaknesses above are genuine and are the basis for the next-step plan carried into the Final Project.